

Final Project Report: Reliability Copilot

Wilson Neira

GitHub Repo Code:

<https://github.com/wilsonnexus/Building-Scalable-Distributed-Systems/tree/main/FinalProject/reliability-copilot>

What was done since previous project draft:

API service on ECS Fargate behind ALB, SQS queue for async job buffering, worker service on ECS Fargate polling SQS, DynamoDB for event/result storage, CloudWatch for logs and metrics, Terraform to provision everything, Locust used to hit the public ALB, and the 3 proposed experiments: bottleneck identification, scaling behavior, and failure injection/recovery.

Smoke test the AWS deployment

Test health:

```
StatusCodes      : 200
StatusDescription : OK
Content          : {"status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 16
                  Content-Type: application/json
                  Date: Wed, 15 Apr 2026 21:28:36 GMT

                  {"status":"ok"}

Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 16], [Content-Type, application/json], [Date, Wed, 15 Apr 2026 21:28:36 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 16
```

Submit one event:

```
$ curl -X POST http://reliability-copilot-alb-2050495761.us-east-1.elb.amazonaws.com/events -H "Content-Type: application/json" -d '{"event_id":"aws-test-1","pipeline_id":"pipe-aws-1","log_text":"timeout while calling dependency service","source":"jenkins","timestamp":"2026-04-14T12:00:00Z"}'
{"event_id":"aws-test-1","status":"queued"}
```

Poll result:

```
$ curl http://reliability-copilot-alb-2050495761.us-east-1.elb.amazonaws.com/results/aws-test-1
{"event_id":"aws-test-1","failure_category":"timeout","recommendation":"Check service latency, retry settings, and downstream availability","risk_score":0.82,"status":"completed","processing_mode":"rule-based-worker"}
```

AWS experiment scripts

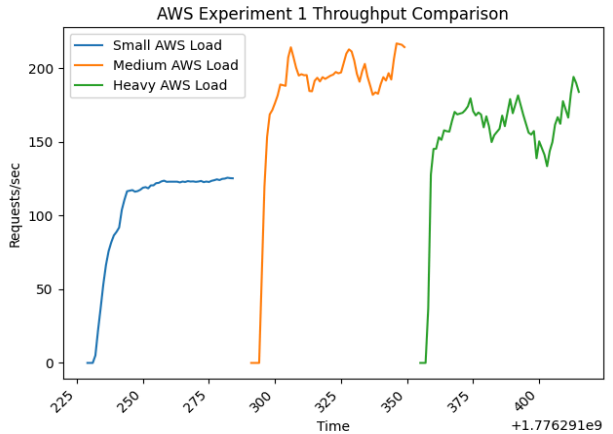
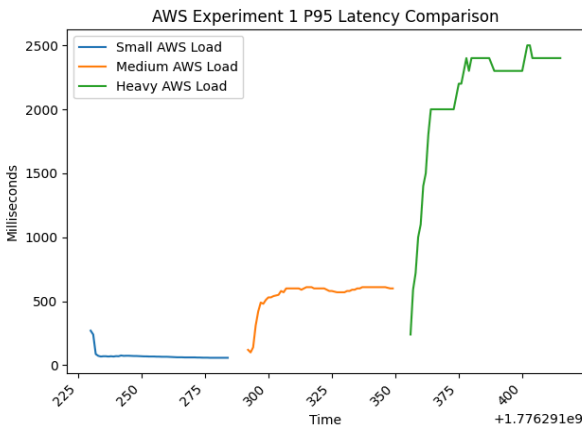
Experiment 1: Bottleneck Identification

Purpose: To identify the first major bottleneck in the system as load increased.

Tradeoff Explored: Throughput versus latency under higher request volume.

Limitations: I used a simplified local deployment and lightweight classification logic, so it does not fully capture cloud-network effects or more expensive inference paths.

Results: Under small AWS load, the system stayed responsive with low latency and stable throughput. Under medium load, latency increased while throughput improved moderately. Under heavy load, latency increased while throughput didn't scale up proportionally. Charts show the system kept accepting requests, response time worsened as concurrency increased.



Analysis: The 1st major bottleneck was not simple request acceptance at the ALB, since the service successfully handled requests, but the downstream processing path became expensive under higher concurrency. The p95 latency graph shows a jump from small to medium to heavy load, throughput doesn't have the same rate increase. Meaning the system was constrained by worker-side processing, queue-backed handling, or storage updates rather than just front door.

Conclusion: System was available under higher load, but performance degraded as load increased, meaning the system needed either more worker capacity, better parallelism, or less per-event processing cost to handle larger bursts well.

Limitations of the analysis: Experiment used lightweight rule-based classification rather than expensive models, the absolute latency numbers would be worse in a realistic AI-heavy version.

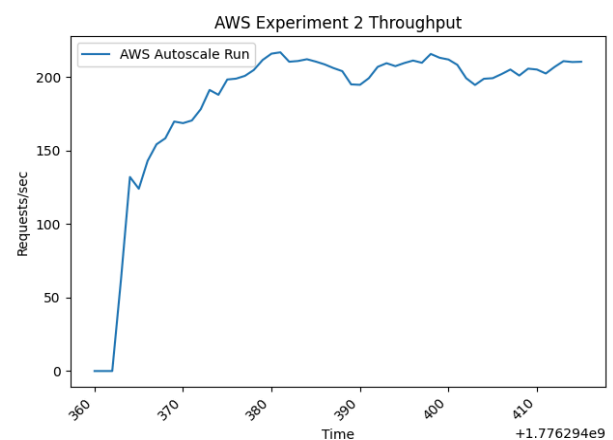
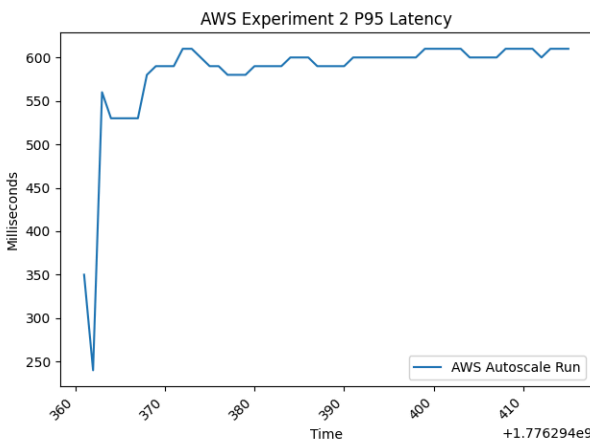
Experiment 2: Horizontal Scaling and Queue-Based Scaling Behavior

Purpose: To test if increasing worker capacity improved system stability & throughput.

Tradeoff Explored: Added worker capacity versus latency and queue pressure.

Limitations: Simulated scaling with worker count changes rather than full production auto scaling.

Results: The autoscale AWS run handled a high number of requests with 0 failures and kept throughput around the low-200 requests/sec range. Latency stayed more controlled than the heavier bottleneck runs, p95 latency staying in a lower band than heavy-load bottleneck cases.



Analysis: Increasing worker-side capacity or improving the processing path helped the system stay stable under sustained load. Compared with heavier bottleneck cases, throughput stayed strong while latency

was controlled, which means the system benefited from having more ability to absorb queued work instead of letting the processing path be the dominant bottleneck.

Conclusion: Added worker capacity improved stability and made the system more scalable. It did not make the system infinitely scalable, but it did reduce the severity of latency growth under load and helped the platform stay responsive at a higher request rate.

Limitations of the analysis: This was not a perfect production-grade autoscaling evaluation because the lab environment and permissions limited how fully I could validate dynamic AWS autoscaling behavior, so it is still closer to a controlled scaling study.

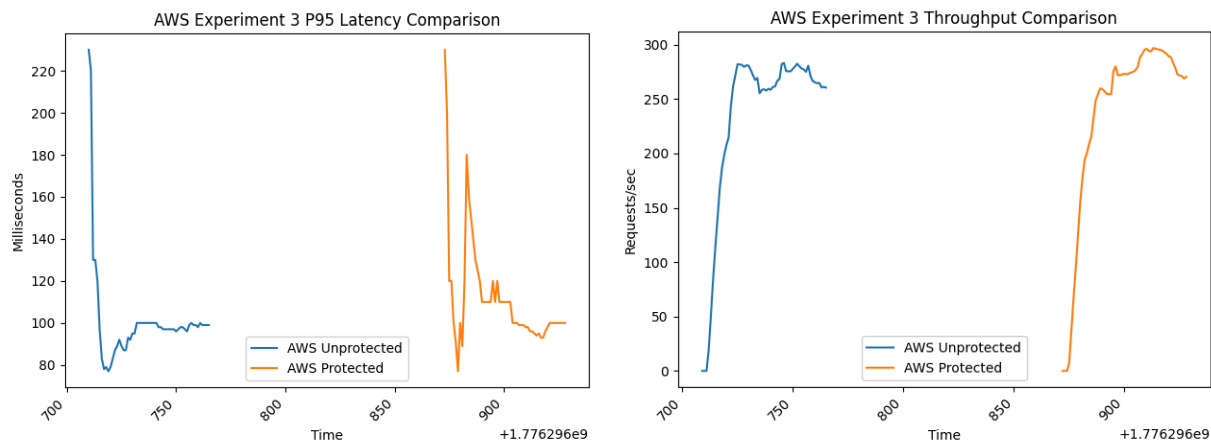
Experiment 3: Failure Injection and Recovery

Purpose: Compare unprotected design with protected design under artificial failure conditions.

Tradeoff Explored: Simplicity versus resilience.

Limitations: Simulated failures rather than coming from a real cloud dependency.

Results: The unprotected AWS failure-injection run, the system showed weaker latency under artificial delays and failures. In the protected run, the system recovered gracefully and kept latency in a tighter range while maintaining strong throughput. The p95 latency and throughput comparisons show that the protected design handled the failure conditions more smoothly.



Analysis: The protected version degraded gracefully under simulated slow inference and injected failures. The p95 latency chart shows a protected path settled into a more stable range, while the throughput chart shows a system could still process requests effectively. Meaning resilience mechanisms helped by reducing the impact level of failure on the rest of the system.

Conclusion: Adding protections like fallback behavior and resilient handling improved the system under failure conditions. The protected version is better as it preserves responsiveness and stability instead of letting failure conditions spread badly through the system.

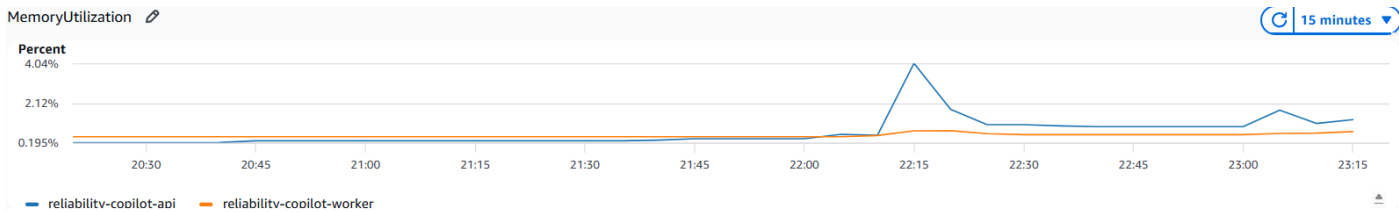
Limitations of the analysis: Failures were simulated and not caused by real external AI endpoints or production database slowdown. The results show the resilience pattern clearly, but they don't fully capture every real-world failure mode.

CloudWatch Metrics

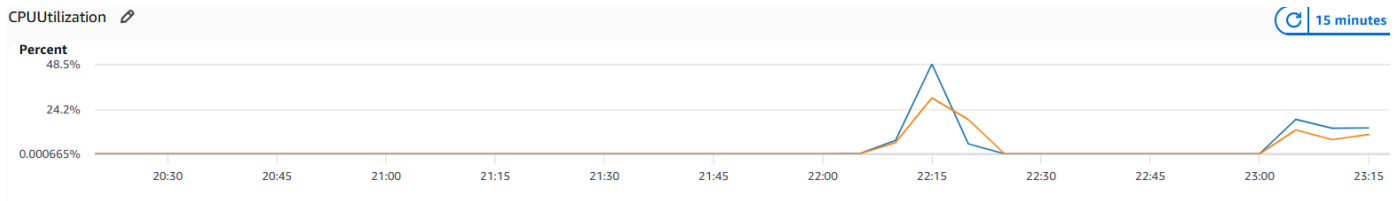
CloudWatch graphs show internal system behaviour during load/failure conditions. ECS CPU/memory utilization show API/worker services were under stress. ALB RequestCount/TargetResponseTime show front-end traffic stayed healthy and response time changed under load. SQS

ApproximateNumberOfMessagesVisible shows queued work started building up, useful for identifying backpressure and worker bottlenecks.

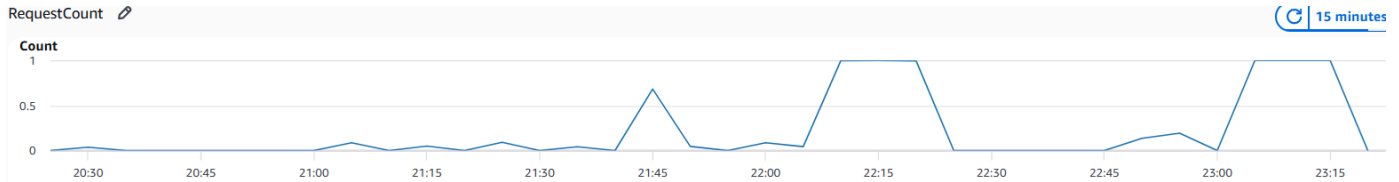
ECS MemoryUtilization



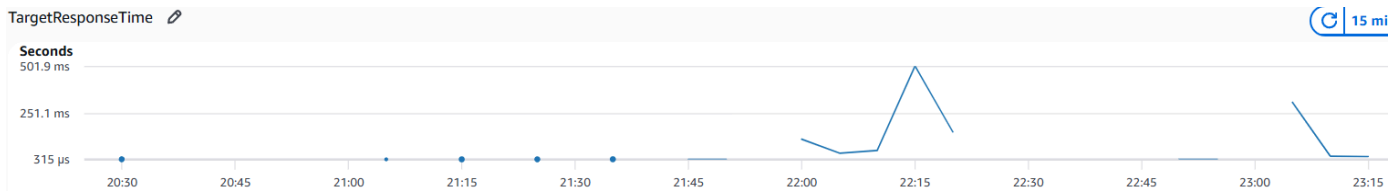
ECS CPUUtilization



ALB RequestCount



ALB TargetResponseTime



SQS ApproximateNumberOfMessagesVisible



Worker Log Evidence: The worker logs show different runtime configurations during the resilience experiment, including unprotected and protected versions. This helps confirm that the failure-injection settings were actually applied in AWS during the experiment runs.

[it](#) > [/ecs/reliability-copilot-api](#) > [ecs/api/20fc22dd0d9549cfb17802078f20c459](#)

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log ev

Timestamp	Message
No older events at this moment. Retry	
2026-04-15T19:58:54.364Z	2026/04/15 19:58:54 API listening on :8080
2026/04/15 19:58:54 API listening on :8080	

t > /ecs/reliability-copilot-worker > ecs/worker/e94debffe0684f67bae7f0cabbe3b2b0

Log events

 **Actions** ▼ 

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#) 

Clear 1m 30m 1h 12h Custom 

▶	Timestamp	Message
No older events at this moment. Retry		
▼	2026-04-15T23:45:30.190Z	2026/04/15 23:45:30 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=false
2026/04/15 23:45:30 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=false		

it > /ecs/reliability-copilot-worker > ecs/worker/b8dfc107aafc4e4887dbc6251d6a53b2

Log events

 **Actions** ▼ 

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#) 

Clear 1m 30m 1h 12h Custom 

▶	Timestamp	Message
No older events at this moment. Retry		
▼	2026-04-15T23:49:16.555Z	2026/04/15 23:49:16 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true
2026/04/15 23:49:16 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true		

t > /ecs/reliability-copilot-worker > ecs/worker/9fb5ad8a6c44481ea43f404727e36c71

Log events

 **Actions** ▼ 

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#) 

Clear 1m 30m 1h 12h Custom 

▶	Timestamp	Message
No older events at this moment. Retry		
▼	2026-04-15T23:51:20.853Z	2026/04/15 23:51:20 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true
2026/04/15 23:51:20 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true		

it > /ecs/reliability-copilot-worker > ecs/worker/45247c0b7325481585d5642c4f22a8fd

Log events

 **Actions** ▼ 

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#) 

Clear 1m 30m 1h 12h Custom 

▶	Timestamp	Message
No older events at this moment. Retry		
▼	2026-04-15T23:54:21.343Z	2026/04/15 23:54:21 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true
2026/04/15 23:54:21 worker starting with WORKERS=1 INFERENCE_DELAY_MS=2000 DB_DELAY_MS=0 FAIL_RATE_PCT=20 PROTECTION_ENABLED=true		